



Vom Papierchaos zur intelligenten Datenbank

Ein Architektur-Pitch für den KI-Kassenbon-Scanner

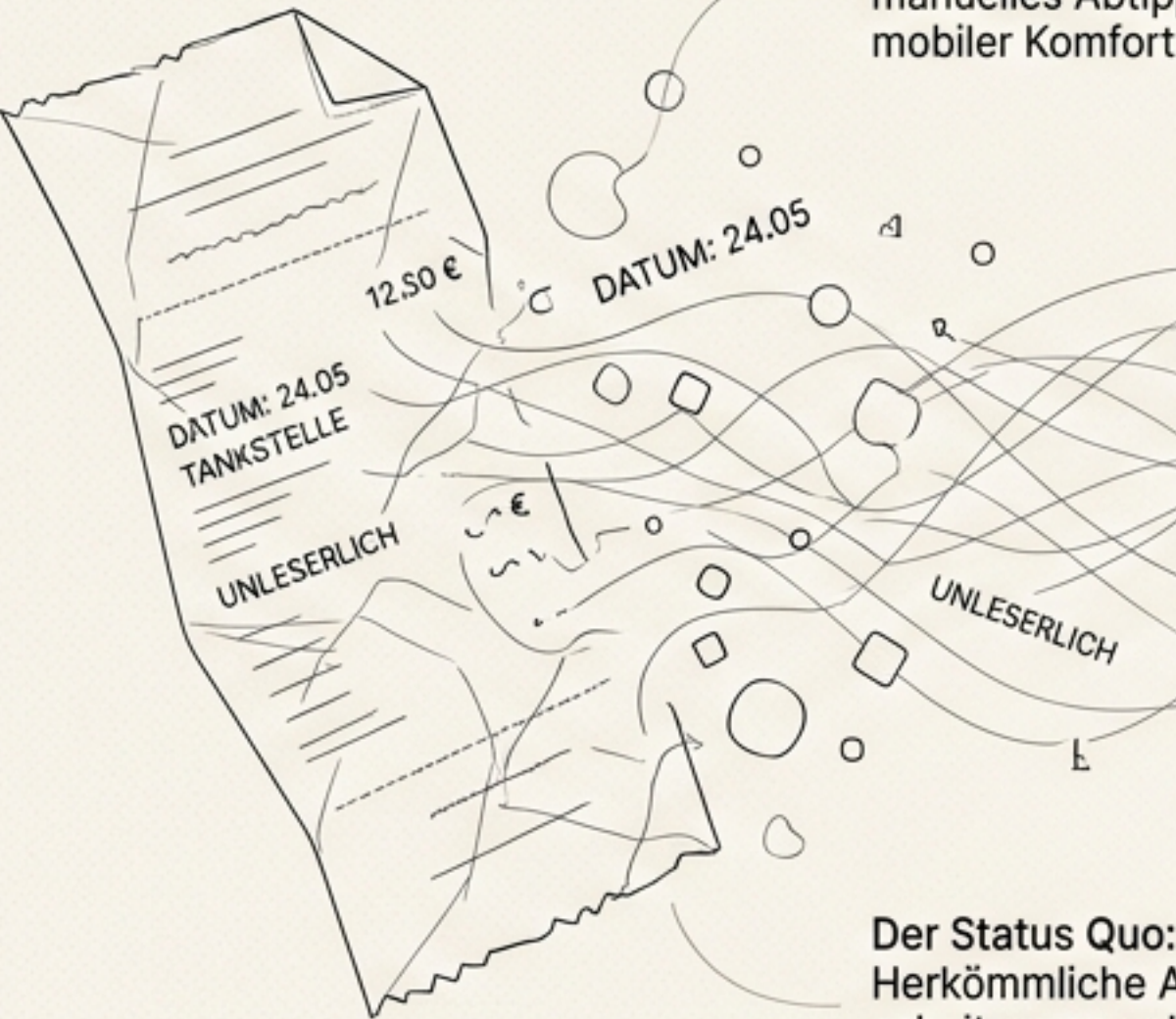
von Safak Özbay

Powered by LangGraph & Dual-OCR

Das Ziel: Ein vollautomatischer Expense-Tracker mit intelligenter Plausibilitätsprüfung.

The Friction

Der Use Case: 100% automatisiertes Expense-Tracking via Telegram. Kein manuelles Abtippen, maximaler mobiler Komfort.



Der Status Quo: Herkömmliche Automatisierungen scheitern an unleserlichen Bons und Randfällen.

Der Clou: LangGraph-Logik

1

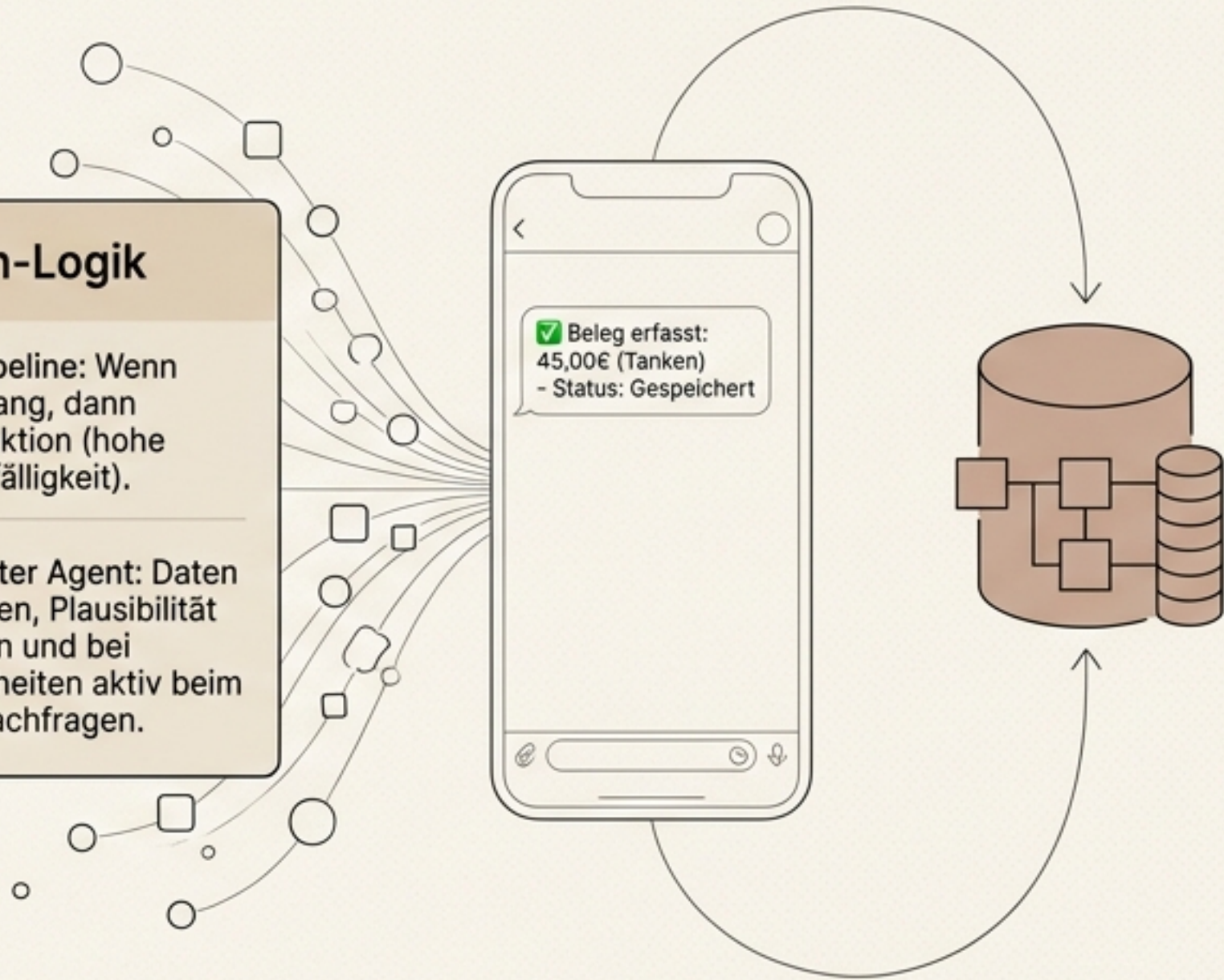
A → X → B

Starre Pipeline: Wenn Bildempfang, dann Textextraktion (hohe Fehleranfälligkeit).

2

Intelligenter Agent: Daten vergleichen, Plausibilität evaluieren und bei Unsicherheiten aktiv beim Nutzer nachfragen.

The Flow



Ein KI-Ökosystem für maximale Präzision und Ausfallsicherheit.



Orchestrierung & Logik

n8n: Dient als zentrale Steuerung und Routing-Ebene.

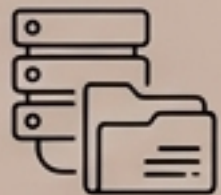
LangChain Code Node: Direkte Integration des LangGraph-Agenten in den Workflow.



Vision & Dual-OCR

GPT-4 Vision: Initiale Klassifizierung (Ist es wirklich ein Kassenbon?).

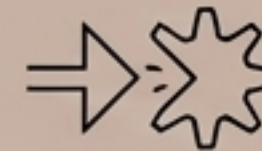
Google Cloud Vision & Azure: Parallele OCR-Verarbeitung für höchste Genauigkeit bei schlechter Bildqualität.



Datenspeicherung

Google Drive: Revisionssicheres Bildarchiv.

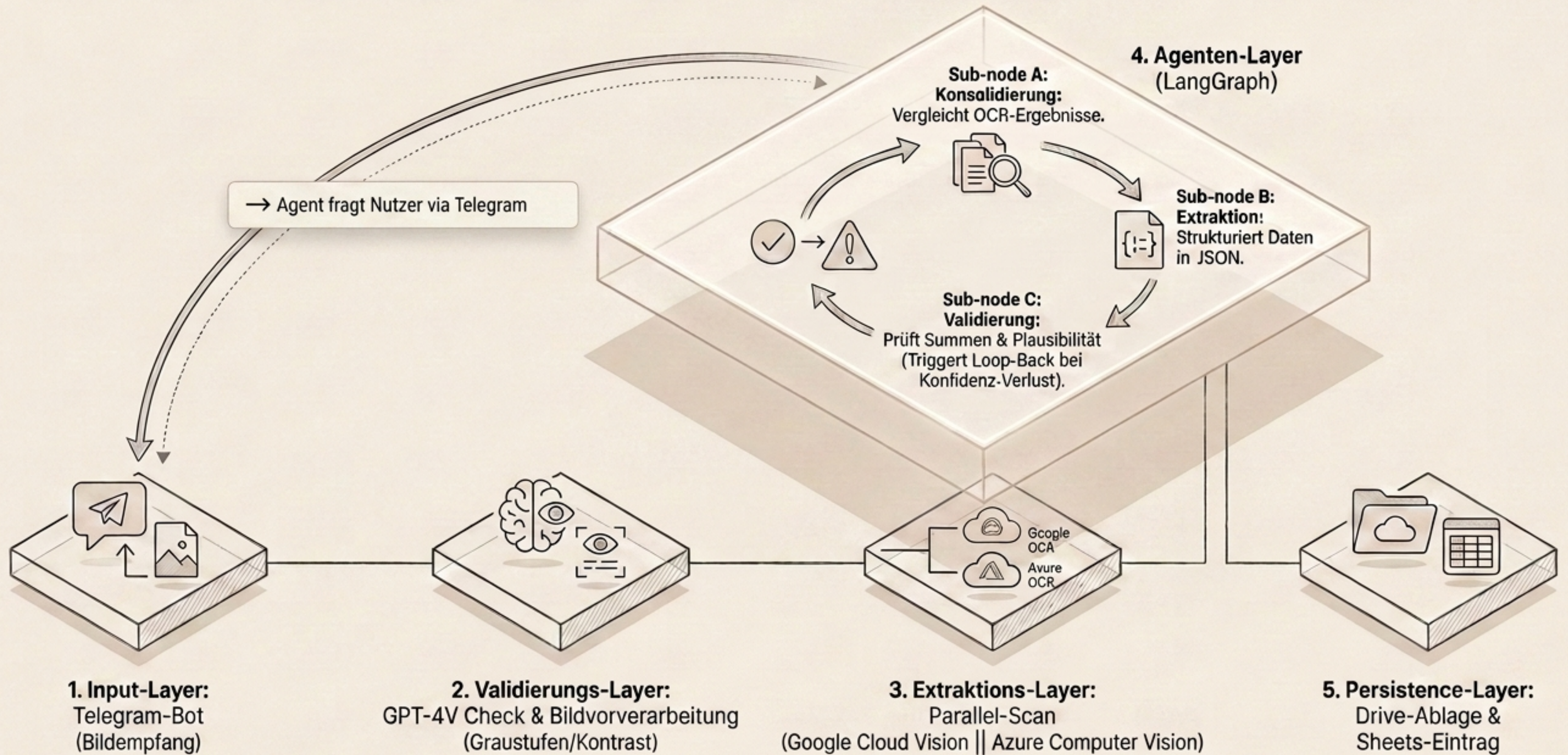
Google Sheets: Strukturierte, relationale Datenbank für Finanzen (inkl. Hyperlinks).



Future Scope (Phase 2)

Memory-Modul: Integration von Langzeitgedächtnis für kontextbasierte Abfragen zu vergangenen Ausgaben via Chat.

Das modulare Flow-Design trennt Extraktion von intelligenter Validierung.



Ein Produkt vier Sprints

Sprint 1: Fundament & Konnektivität

Einrichtung Telegram-Bot & n8n-Workflow.
Anbindung von Google Drive & Sheets für die
Basisspeicherung.
Implementierung der Bildvalidierung via GPT-4 Vision.

Sprint 2: Vision & OCR-Basis

Integration der ersten OCR-Engine (Google Vision).
Einbau der Azure OCR für den Dual-Scan.

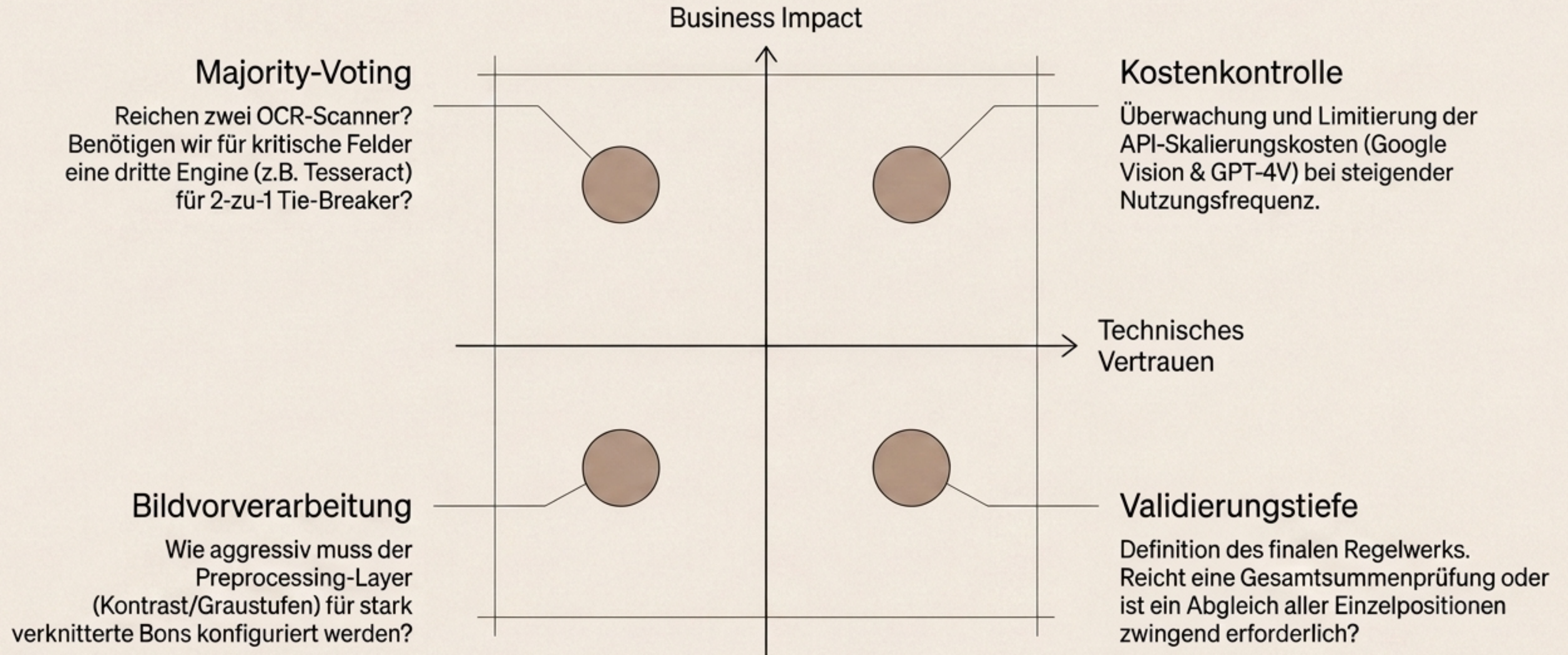
Sprint 3: Intelligenz & Orchestrierung

Aufbau des LangGraph-Systems (Vergleichs- und
Extraktions-Agenten). Implementierung des
Validierungs-Agenten (Plausibilitätschecks).

Sprint 4: Autonomie & Feedback

Finalisierung der interaktiven Telegram-
Statusmeldungen (Loop-Back).

Risikos, Randfälle usw.



The background features a network of thin, light gray lines connecting several 3D polyhedral shapes. These shapes, which resemble truncated octahedrons, are rendered in two shades of brown: a darker, muted brown and a lighter, more vibrant brown. They are positioned at various points across the frame, creating a sense of interconnectedness and complexity.

Sprint 2: Datenextraktion & Persistenz

Vom erkannten Kassensbon zur
strukturierten Datenbank.

Expense Tracker – Sprint Review

Die neuen Kernkomponenten im System



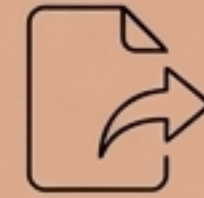
Präzise Extraktion

Wechsel auf Keyvan/german-ocr-3 (basiert auf Qwen3.5 8b).
Sprung von reiner Bildklassifizierung zur spezialisierten, strukturierten JSON-Ausgabe für Rechnungen.
Parst die Daten direkt als lesbaren Telegram-Text via Format-Node.



Robuste Persistenz

Lokale Bildspeicherung im Verzeichnis (/home/node/.n8n/expenseTracker/bin/).
Strukturierte Datensicherung im atomaren JSONL-Format (receipts_YYYY.jsonl), welches multi-user-sicher ist.



User-Interaktion

Neues Textkommando /export [Jahr] im Telegram-Bot.
Generiert User-isolierte (gefiltert nach chatId), Excel-Excel-kompatible (Semikolon-getrennte) CSV-Exporte direkt als Telegram-Dokument.

Design-Entscheidungen & Workarounds

Speichermedium

Herausforderung

n8n Sandbox blockiert native Module (z.B. better-sqlite3 Binding-Fehler).

Gewählte Architektur

JSONL statt SQLite

Verzicht auf relationale DBs zugunsten von einfachen Code-Nodes und strukturierten Textdateien.

Parallelität

Herausforderung

Parallele Telegram-Anfragen verursachen Race Conditions beim klassischen Read-Modify-Write.

Gewählte Architektur

Atomares Schreiben

Nutzung von `fs.appendFileSync`, um Daten sicher an die Liste anzuhängen, ohne bestehende Datensätze zu überschreiben.

Dateizugriff

Herausforderung

Strikte n8n File-Access Limits beim Speichern von Originalbildern.

Gewählte Architektur

Code-Nodes statt Standard-Module

Umgehung der Limits durch direktes Speichern via `fs.writeFileSync` anstelle der nativen `WriteBinaryFile-Node`.

Roadmap & offene Baustellen



Interaktiver KI-Agent

Natürliche Sprachverarbeitung für dynamische Abfragen (z.B. 'Wie viel habe ich letzten Monat für Tanken ausgegeben?') direkt im Chat.

Artikel-Kategorisierung

Intelligente, automatische Zuordnung der extrahierten Ausgaben in fixe Budget-Kategorien.

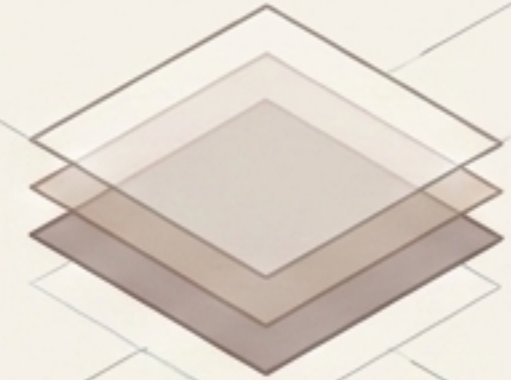
CSV-Export patchen

Aktuelle CSV-Struktur ist noch fehlerhaft und muss für den nahtlosen Excel-Import bereinigt werden.

Bugfix & Foundation

Next Feature

Vision



Sprint 3: Von der Extraktion zur intelligenten Konversation.

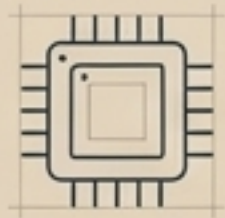
Das System lernt sprechen. Fokus auf die Implementierung des KI-Agenten für natürlichsprachige Abfragen und tiefgreifende System-Optimierung.

Das Ziel: Ein interaktiver Finanz-Assistent, nicht nur ein Datengrab.

Der Fokus: Natürliche Sprachverarbeitung (NLP) & System-Robustheit.

Der Status: Erfolgreich implementiert.

Die Anatomie unseres neuen AI-Agenten



Säule 1

LLM Engine (Lokal & Präzise)

Integration von Ollama mit dem Modell Keyvan/german-text-3.1. Maximale Datenkontrolle und herausragendes deutsches Sprachverständnis.



Säule 2

Orchestrierung & Tool-Calling

n8n LangChain Agent als Herzstück. Fähigkeit zum autonomen Tool-Calling (spezifisch: search_expenses), um dynamisch Ausgaben in der Datenbank nachzuschlagen.



Säule 3

Kontext & Gedächtnis

Window Buffer Memory: Der Agent erinnert sich an den Verlauf der Konversation.
Reply-Kontext-Support: Nahtloses Anknüpfen an spezifische Nachrichten oder Belege im Telegram-Chat.

... Wie viel für Tanken?

Memory
Check

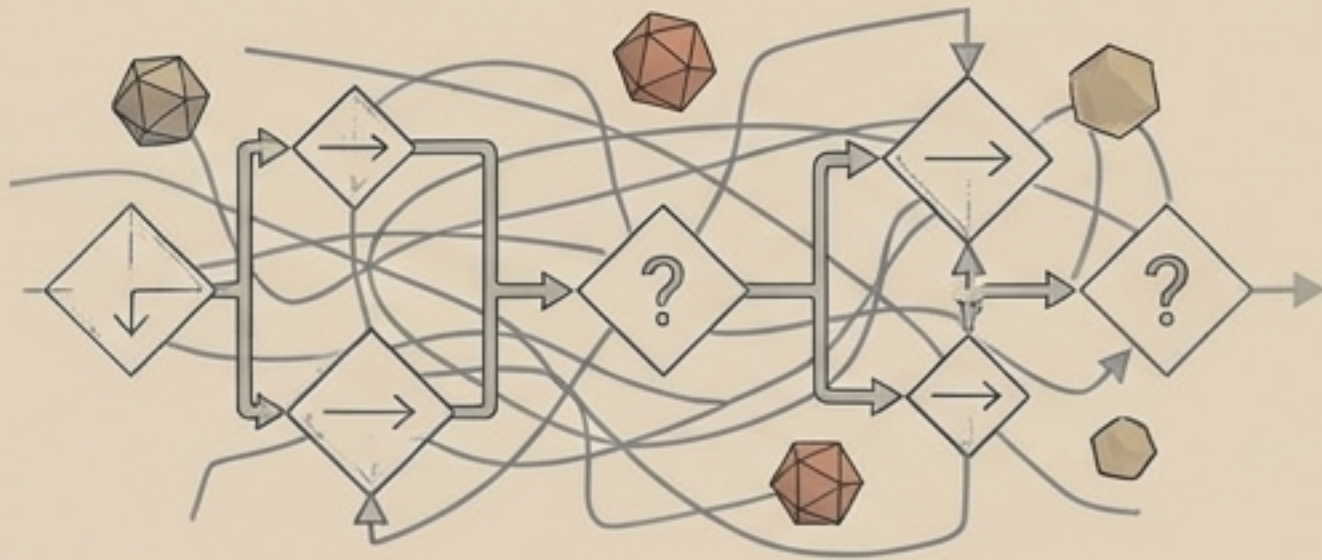
Tool Call

Ollama
Formatting

Du hast diesen Monat
45,00€ fürs Tanken
ausgegeben.

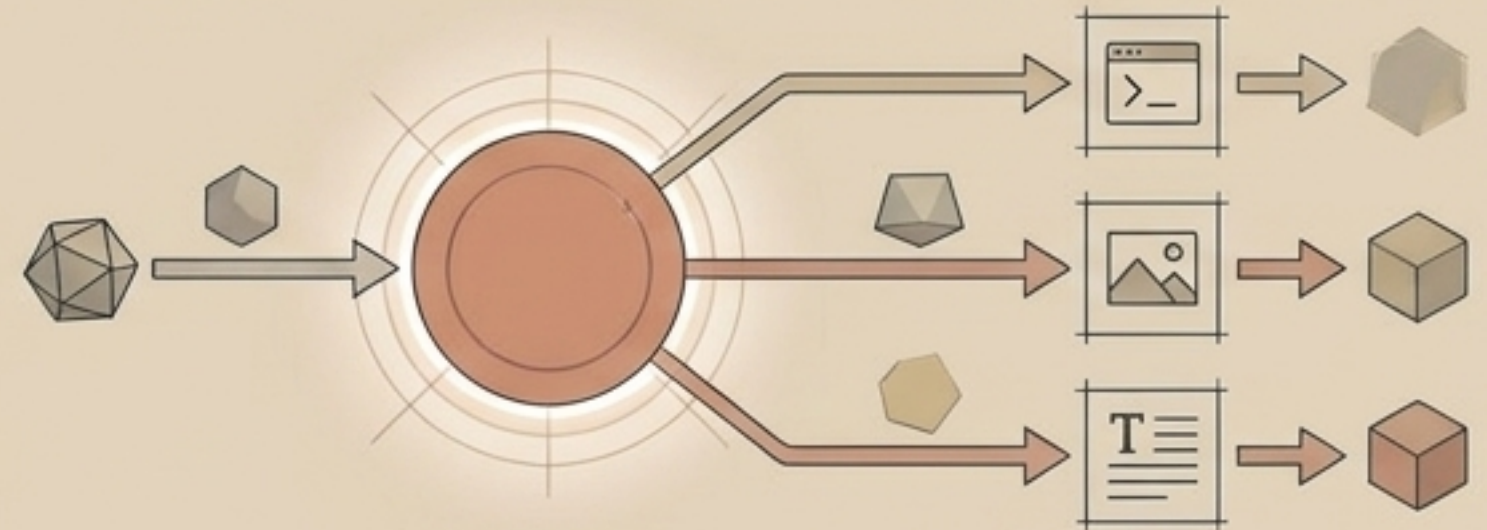
Architektur-Refactoring für Skalierbarkeit

Status Quo (Sprint 2): Verkettete IF-Nodes



- Hohe Komplexität.
- Schwer zu warten und zu erweitern.
- Fehleranfälliges Routing.

Neue Architektur (Sprint 3): Zentraler Switch-Node



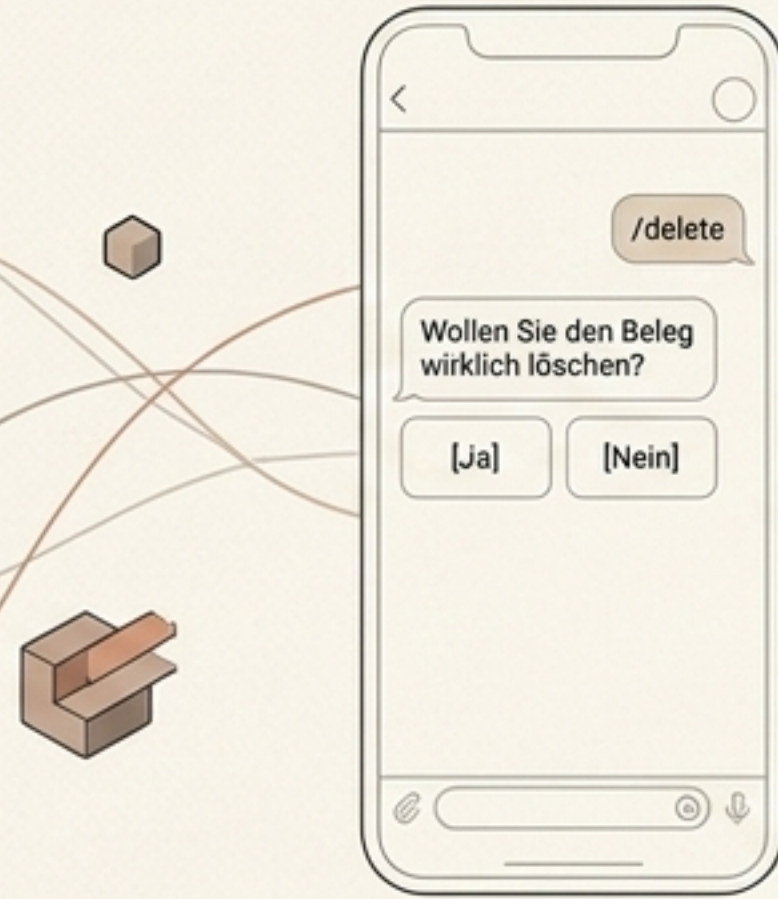
- Single Source of Truth für das Routing von Telegram-Inputs.
- Saubere Trennung von Befehlen, Bildern und Text.
- Bereit für beliebig viele zukünftige Features.



Das Schutzschild: Global Error Handler

Ein neuer, dedizierter Workflow, der Fehler aus allen Sub-Workflows zentral abfängt, loggt und dem Nutzer nutzbare Feedback-Meldungen sendet. Schluss mit Silent Fails.

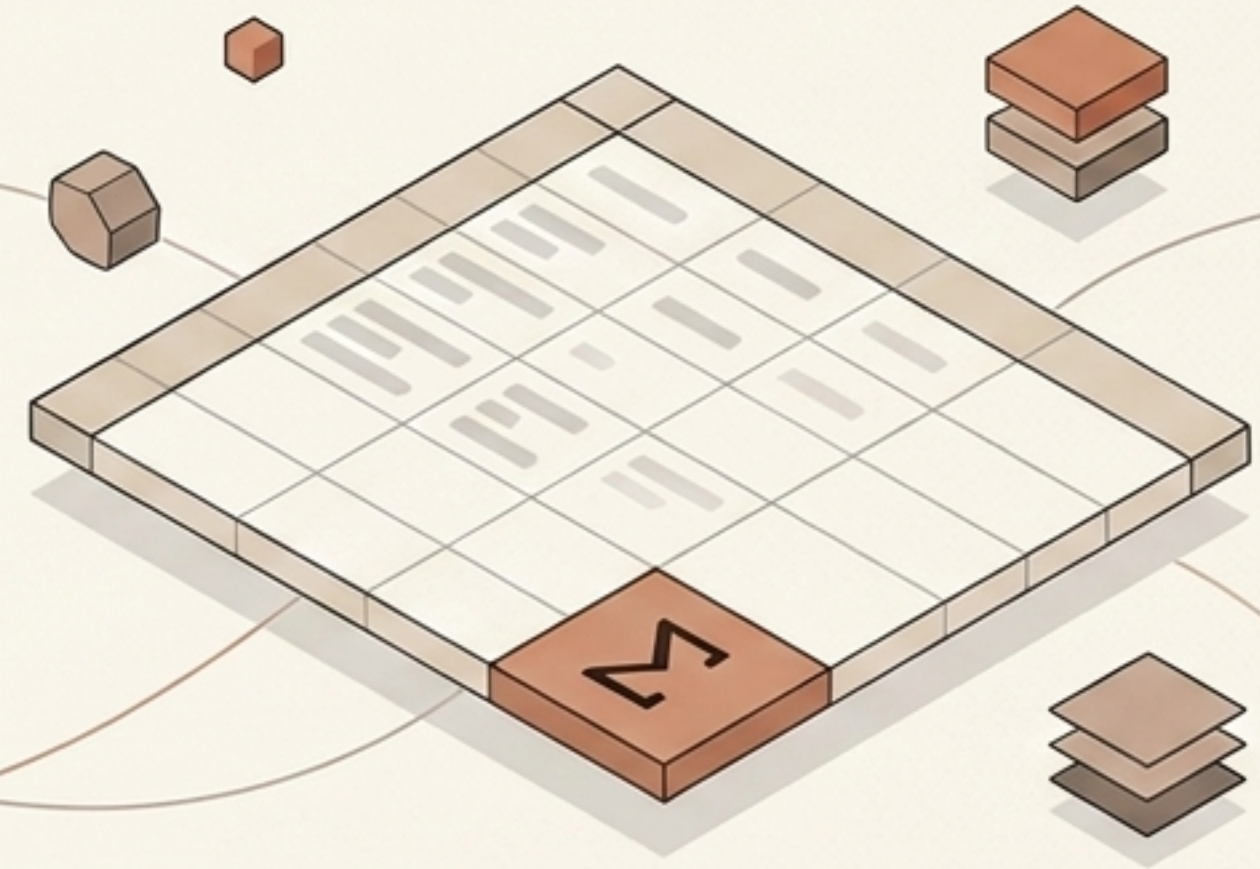
Mehr Kontrolle. Bessere Daten.



Telegram Command-Center

/list: Schneller, kompakter Überblick der letzten erfassten Ausgaben direkt im Chat.

/delete: Neues Lösch-Kommando mit Sicherheits-Feature (integrierter Bestätigungsschritt via Inline-Keyboard zur Vermeidung von Unfällen).

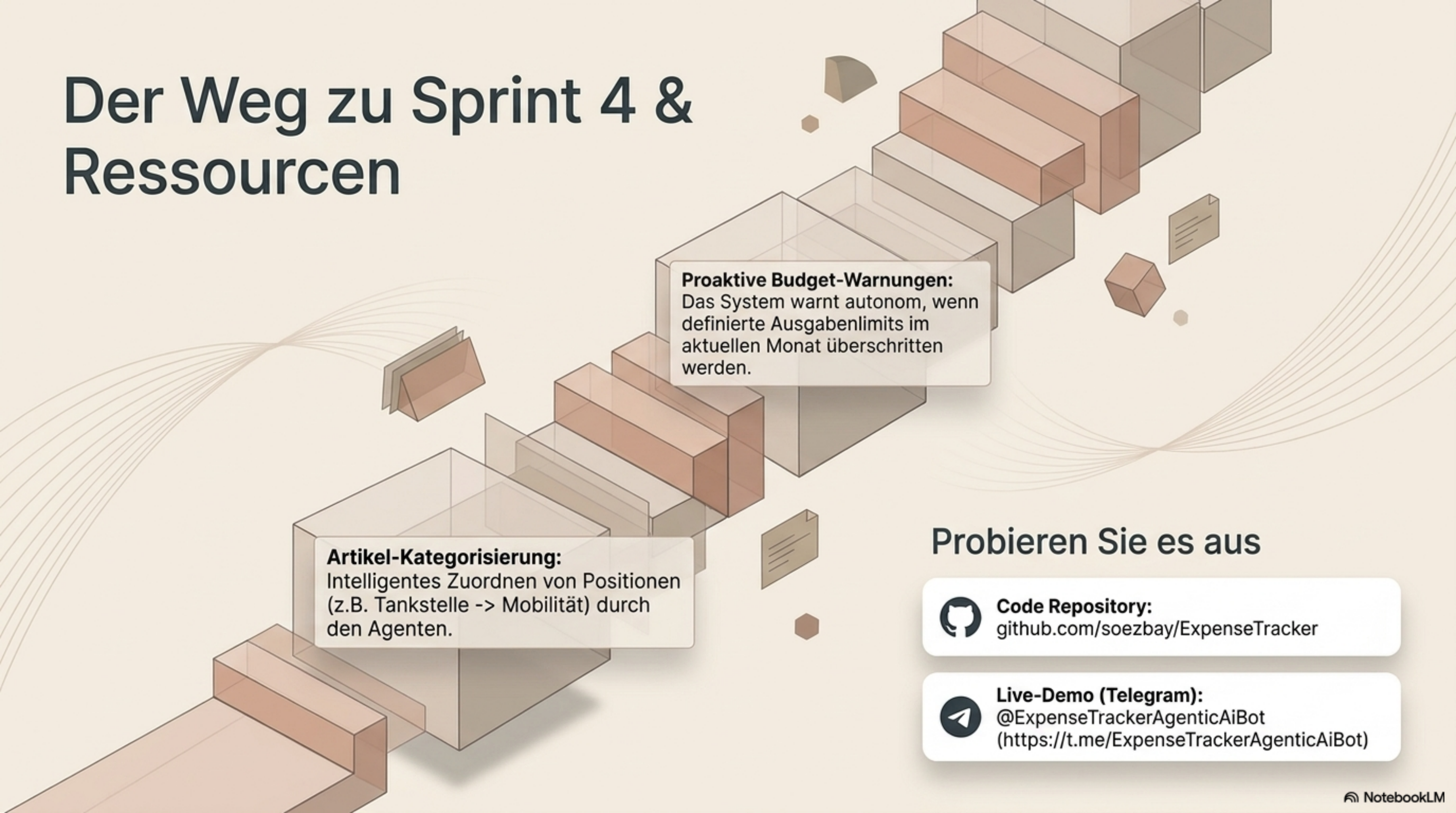


Der CSV-Export Patch

Standard-Konformität: Komplettes Rewrite auf den offiziellen RFC 4180 Standard. Behebt Parsing-Fehler beim nahtlosen Import in Excel.

Smart Data: Automatische Summenberechnung der exportierten Ausgaben ist nun direkt in der generierten Datei integriert.

Der Weg zu Sprint 4 & Ressourcen



Proaktive Budget-Warnungen:
Das System warnt autonom, wenn definierte Ausgabenlimits im aktuellen Monat überschritten werden.

Artikel-Kategorisierung:
Intelligentes Zuordnen von Positionen (z.B. Tankstelle -> Mobilität) durch den Agenten.

Probieren Sie es aus



Code Repository:
github.com/soezbay/ExpenseTracker



Live-Demo (Telegram):
[@ExpenseTrackerAgenticAiBot](https://t.me/ExpenseTrackerAgenticAiBot)
(<https://t.me/ExpenseTrackerAgenticAiBot>)

Kassenbons nerven – wir machen sie gesprächig

Vom Zettelchaos zum smarten, lokalen Finanz-Agenten in Telegram

Excel-Chaos



Foto-Upload



AI-Agent



100% Lokal

Vom Foto-Upload zur intelligenten Finanzanalyse in natürlicher Sprache

Erfassen



OCR-Extraktion
(Händler, Betrag, MwSt)

“
Bot: „Rewe, 42,30 €,
Lebensmittel, gespeichert.“

Verstehen



Natürlichsprachliche
Abfragen

“
User: „Wie viel habe ich im Juni
ausgegeben?“
-> Bot: „14 Belege, 623,80 € gesamt.“

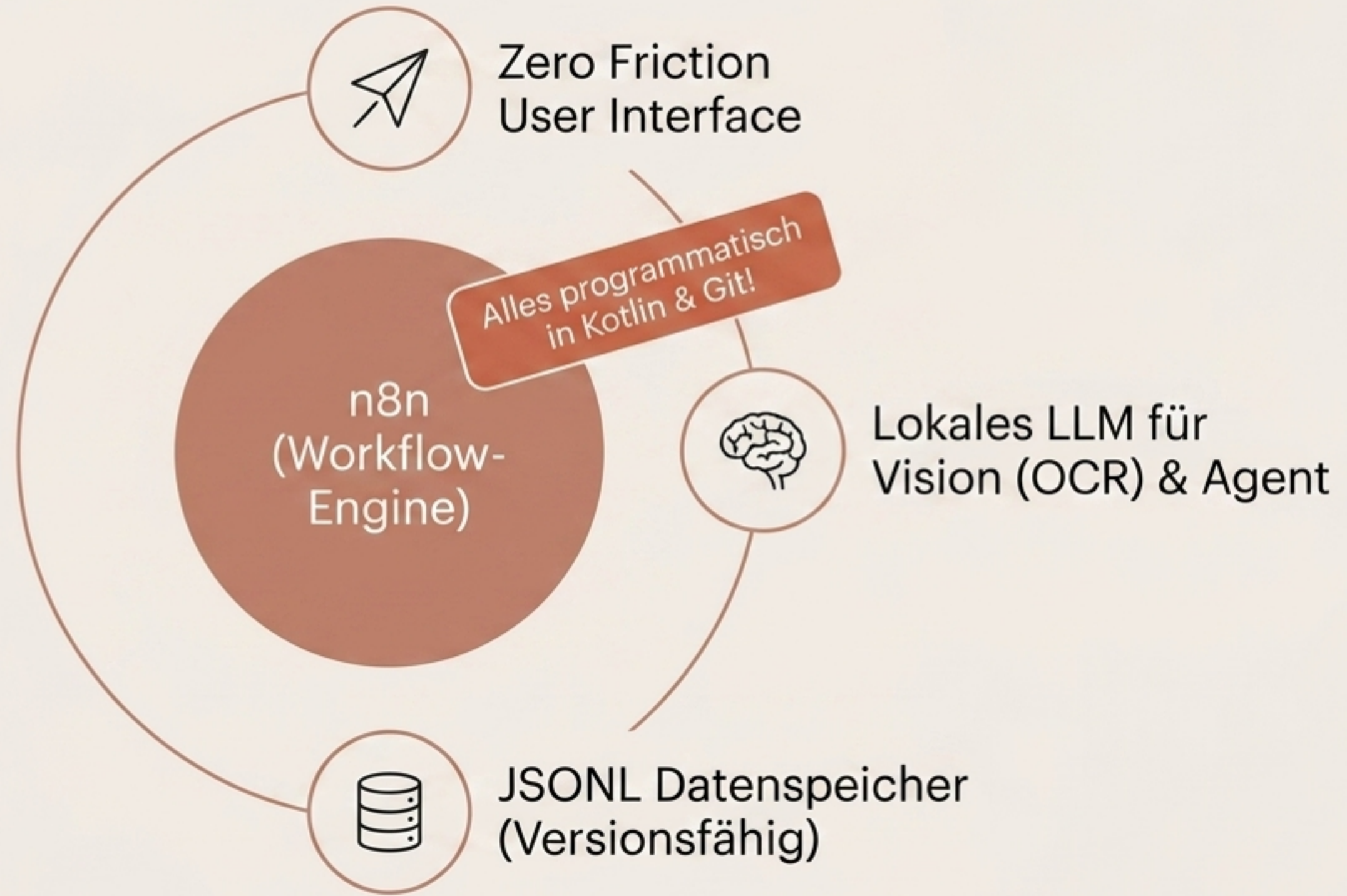
Strukturieren



Regelbasierte Kategorisierung
& Excel-Export

“
User: /export 2026
-> Bot generiert XLSX-Datei.

Ein modulares, lokales KI-Ökosystem trennt Extraktion von Validierung



Intelligente Agenten schlagen starre Automatisierungs-Pipelines

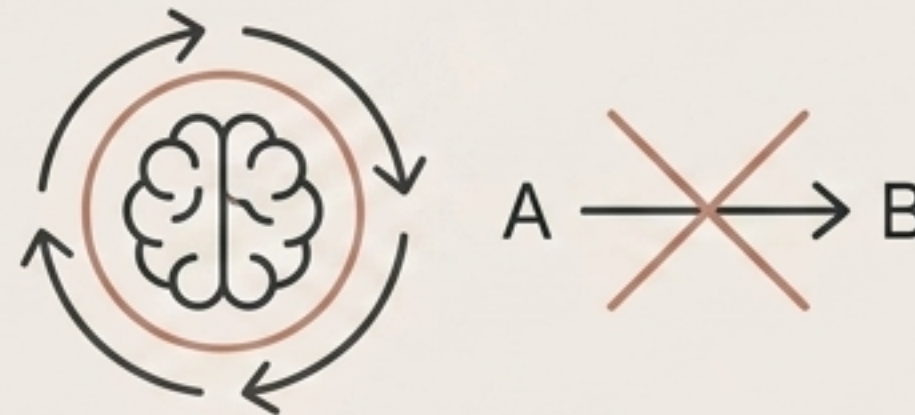
100% Lokal & Privat

**Lokale LLMs statt
Cloud-APIs.**

Autonau LLMs & Privat.

Volle Datensouveränität über
Finanzdaten, keine laufenden API-
Kosten pro Beleg.

Tool-Calling Agent



Dynamische Entscheidungsfindung
statt Wenn-Dann-Regeln.

Agent wählt autonom zwischen
`get_summary_stats`, `compare_periods`
oder `category_breakdown`.

Intelligente Kategorisierung

**Regelbasiertes
Zuweisungs-Regelwerk.**

Regelbasiertes
Zuweisungs-Regelwerk.

Blitzschnell, transparent und sofort
anpassbar ohne teures LLM-
Neutraining.

Von der JSONL-Datei zum proaktiven, relationalen Finanzassistenten

Learnings

Prompt-Fragilität: Bei OCR entscheidet ein einziges Wort im Prompt über Erkennung oder Blindflug.

Präzision: Tool-Calling-Agents benötigen absolut wasserdichte Tool-Beschreibungen.

Evaluation: Ein Framework zur messbaren Bewertung von Prompt-Änderungen ist unerlässlich.

Roadmap

- **Infrastruktur:** Umstieg von JSONL auf Postgres/SQLite für komplexe Aggregationen.
- **Skalierung:** Multi-User-Trennung und dediziertes Web-UI Dashboard.
- **Proaktivität:** Automatische Budget-Mahnungen bei wiederkehrenden Zahlungen.

Probiert es aus – der Bot wartet auf Telegram!